C#.NET Programming Concepts

Topic: OOPs, Principles, Constructors, Properties & Access Modifiers

1. Introduction to Object-Oriented Programming (OOP)

What is OOP?

Object-Oriented Programming (OOP) is a programming approach where programs are built using **objects** instead of only functions.

An **Object** represents a real-world entity that contains:

- **Data (Properties/Fields)** → Characteristics
- **Behavior (Methods)** → Actions

Real-world Example

Car

Properties:

- Brand
- Color
- Speed

Methods:

- Start()
- Stop()
- Accelerate()

Why OOP?

OOP helps developers:

- Write reusable code
- Reduce code duplication
- Improve security
- Make applications easier to maintain
- Organize large applications

OOP in C#

Everything in C# is based on **Classes** and **Objects**.

Example:

```
class Student
{
   public string Name;

   public void Display()
   {
      Console.WriteLine(Name);
   }
}
Creating an object:
Student s = new Student();

s.Name = "John";

s.Display();
```

2. Four Principles of OOP

There are four major principles.

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

2.1 Encapsulation

Definition

Wrapping data and methods together into a single unit (Class).
It also hides data using access modifiers.
Example

```
class Employee
{
   private int salary;

   public void SetSalary(int s)
   {
      salary = s;
   }

   public int GetSalary()
   {
      return salary;
   }
}
```

2.2 Abstraction

Definition

Showing only necessary information while hiding internal implementation.

Example

```
class ATM
{
   public void Withdraw()
   {
      Console.WriteLine("Cash Withdrawn");
   }
}
```

User only sees
Withdraw()
Internal banking logic is hidden.

2.3 Inheritance

Definition

Inheritance allows one class to reuse another class's members.
Syntax
class Parent

```
{

}

class Child : Parent
{

}
```

Example

```
class Animal
{
   public void Eat()
   {
      Console.WriteLine("Eating");
   }
}

class Dog : Animal
{

}
Usage
Dog d = new Dog();

d.Eat();
```

2.4 Polymorphism

Definition

One method behaves differently in different situations.

Types

- Compile-time (Method Overloading)
- Run-time (Method Overriding)

Example (Overloading)

```
class Calculator
{
   public int Add(int a, int b)
   {
      return a + b;
   }

   public int Add(int a, int b, int c)
   {
      return a + b + c;
   }
}
```

Example (Overriding)

```
class Animal
{
   public virtual void Sound()
   {
```

```
        Console.WriteLine("Animal Sound");
    }
}

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Bark");
    }
}
```

OOP Principles Summary

Principle	Meaning
Encapsulation	Hide data
Abstraction	Hide implementation
Inheritance	Reuse code
Polymorphism	One interface, many forms

3. Constructors

What is a Constructor?

A constructor is a special method that executes automatically when an object is created.

Characteristics

- Same name as class
- No return type
- Called automatically

Example

```
class Student
{
    public Student()
    {
        Console.WriteLine("Object Created");
    }
}
Student s = new Student();
```

Output

Object Created

Why Constructors?

- Initialize objects
- Set default values
- Allocate resources

Types of Constructors

---

1. Default Constructor

```
class Student
{
   public Student()
   {
      Console.WriteLine("Default Constructor");
   }
}
```

---

2. Parameterized Constructor

```
class Student
{
   string name;

   public Student(string n)
   {
      name = n;
   }

   public void Display()
   {
      Console.WriteLine(name);
   }
}
Usage
Student s = new Student("David");
s.Display();
```

Constructor Overloading

Multiple constructors in the same class.

```
class Employee
{
   public Employee()
   {

   }

   public Employee(string name)
   {

   }

   public Employee(int id)
   {

   }
}
```

4. Properties

What is a Property?

A property provides controlled access to private fields using **get** and **set** accessors.

Without Property

```
class Student
{
   public string Name;
}
```

Better approach

```
class Student
{
   private string name;

   public string Name
   {
      get
      {
         return name;
      }

      set
      {
         name = value;
      }
   }
}
```

Usage

```
Student s = new Student();

s.Name = "John";

Console.WriteLine(s.Name);
```

Auto-Implemented Property

public string Name { get; set; }

This automatically creates the private field.

Read-only Property

public string Name { get; }

Benefits of Properties

- Data validation
- Encapsulation
- Easy maintenance
- Better readability

5. Access Modifiers

What are Access Modifiers?

Access modifiers define **who can access a class or its members**.

Types of Access Modifiers

1. Public

Accessible from anywhere.

```
public class Student
{
   public string Name;
}
```

---

2. Private

Accessible only inside the same class.

```
private int salary;
```

---

3. Protected

Accessible inside the class and derived classes.

```
protected int age;
```

---

4. Internal

Accessible within the same project (assembly).

```
internal class Employee {
}
```

---

5. Protected Internal

Accessible within the same assembly or from derived classes in another assembly.

```
protected internal int marks;
```

Access Modifier Diagram

```
Public
  |
  ├── Accessible Everywhere

Internal
  |
  ├── Same Project

Protected
  |
  ├── Same Class
  └── Child Class

Private
  |
  └── Same Class Only
```

Access Modifier Summary

Modifier	Same Class	Derived Class	Same Project	Other Projects
Public	✓	✓	✓	✓
Private	✓	✗	✗	✗
Protected	✓	✓	✗	✗
Internal	✓	✓	✓	✗
Protected Internal	✓	✓	✓	✓ (derived classes)

Key Takeaways

- **OOP** organizes programs using **classes** and **objects**.
- The four OOP principles are **Encapsulation**, **Abstraction**, **Inheritance**, and **Polymorphism**.
- **Constructors** initialize objects automatically when they are created.
- **Properties** provide controlled access to private data using get and set.
- **Access Modifiers** determine the visibility and accessibility of classes and their members.
- Using OOP concepts correctly leads to **reusable, secure, maintainable, and scalable** C# applications.